OKOK，经过前面的内容，接下来我们又到了一个高峰，字节码解析，虚拟机肯定不是凭空跑起来的，肯定是有代码才会去执行

这一篇我们就来进行字节码部分代码的分析

还是我们的 `lvm.c` 这个文件，还是虚拟机的分发器，不过这次看的是 `GET_OPCODE` 这个是一个宏定义

```
#define GET_OPCODE(i) (cast(OpCode, ((i) >> POS_OP) & MASK1(SIZE_OP, 0)))

#define POS_OP 0 // 操作码在指令中的起始位置
#define SIZE_OP 7 // 指令长度
#define MASK1(n, p) ((~((~(Instruction)0) << (n))) << (p))
#define cast(t, exp) ((t)(exp))
typedef unsigned int l_uint32;
typedef l_uint32 Instruction;
```

1. `MASK1(n, p)` 拆解

```
MASK1(n, p) ((~((~(Instruction)0) << (n))) << (p))
```

- `~(Instruction)0` → 全 1（二进制全部置 1），比如 32 位就是 `0xFFFFFFFF`
- `(~(Instruction)0) << n` → 左移 n 位，高位还是 1，低 n 位变成 0
- `~((~(Instruction)0) << n)` → 再取反，高位 n 位变 1，其余 0
- `<< p` → 把这段连续的 1 左移 p 位

所以 `MASK1(n, p)` 的效果就是：

生成一个长度为 n 的连续 1 位的掩码，然后整体左移 p 位，下面来个小案例

```
MASK1(7, 0)
= ((~((~0) << 7)) << 0)
= (~(0xFFFFFFFF << 7)) << 0
```

```
= (~0xFFFFFF80) << 0
= 0x7F
```

2.GET_OPCODE 分析

- `i >> POS_OP` → 右移操作码在指令中的位置（Lua 里 `POS_OP=0`，所以不动）
- `& MASK1(SIZE_OP, 0)` → 只保留低 `SIZE_OP` 位（这里是 7 位）
- `cast(OpCode, ...)` → 把结果转换成 `OpCode` 类型（通常是枚举）

  所以这段宏最终做的事情就是：

> ✨ 从 Lua 指令 `i` 里提取操作码（Opcode）字段，Lua 里操作码占低 7 位

更像人看的代码的话差不多就是，下面这种格式

```
OpCode op = (OpCode)(instr & 0x7F);
```

OpCode 是枚举类型的数组，C 会隐式转化为 int 类型，直观感受就差不多相当于数组偏移，我自己也打印出来几条指令集

```
instr=0x00000053
instr=0x00000001
instr=0x0000000B
```

第一个是第 83 条，第二个就是第一条，第三个就是 11 条

总结

Lua 字节码是一个 32 位整数，最低 7 位存放操作码。宏 `GET_OPCODE(i)` 的作用就是从指令 `i` 中提取操作码，用于虚拟机分发。它通过掩码 `MASK1` 保留操作码位，再转换成 `OpCode` 枚举类型，直观理解就是：

```
OpCode op = (OpCode)(instr & 0x7F);
```

这样虚拟机就知道当前要执行哪条指令，为指令表查找和调度做准备。